Práctica Final. Servicio de envío de mensajes (Partes 1 y 2)

Sistemas Distribuidos



Autor 1	Autor 2
Nombre: Pablo Pino Castillo **NIA:** 100522129 **Email:** 100522129@alumnos.uc3m.es	**Nombre:** Pablo Pérez Tenorio **NIA:** 100522321 **Email:** 100522321@alumnos.uc3m.es

Índice de contenidos

1. Diseño de la aplicación **2**
1.1 Servidor (server.c) 3
1.2 Cliente (client.py) 4
1.3 Servicio Web (ws-conversor.py) 6
1.4 RPC 6
2. Compilación y ejecución **7**
2.1 Compilación del servidor 7
2.2 Ejecución del sistema 7
3. Batería de Pruebas y Resultados **8**
P1 — REGISTER / UNREGISTER 8
P2 — CONNECT / DISCONNECT 9
P3 — SEND con destinatario conectado 9
P4 — USERS / CONNECTED USERS 10
P5 — Concurrencia: múltiples clientes simultáneos 10
P6 — SENDATTACH + GETFILE 11
P7 — SENDATTACH con destinatario offline 11
P8 — Normalización de mensajes (Web Service SOAP) 12
P9 — RPC Logging y tolerancia a fallos 12
P10 — Robustez y casos límite 13
4. Conclusiones **13**

1. Diseño de la aplicación

En esta sección se describen las principales funciones implementadas en cada uno de los módulos que componen la práctica (sin incluir el código fuente).

Para organizar el estado del servidor y la comunicación entre hilos, se han definido tres estructuras propias en **almacen.h.**

struct mensaje: representa un mensaje almacenado en la cola de pendientes de un usuario. Contiene el identificador único del mensaje (id), el nombre del remitente (remitente), el contenido del mensaje (contenido) y el nombre del fichero adjunto (fichero), este último solo relevante en mensajes de tipo SENDATTACH. Se usa como tipo de los elementos del array mensajes_pendientes dentro de struct info_cliente, permitiendo que el servidor guarde los mensajes que no han podido entregarse porque el destinatario estaba desconectado.

struct info_cliente: representa el estado completo de un usuario registrado en el sistema. Almacena su nombre (nombre), su estado de conexión (estado, 0 o 1), la dirección IP y el puerto de escucha cuando está conectado (IP, PUERTO), un contador de mensajes enviados que se usa como generador de identificadores únicos (contador_mensajes), y la cola de mensajes pendientes de entrega junto con su tamaño (mensajes_pendientes[], num_mensajes_pendientes). El servidor mantiene un array global de estas estructuras (clientes[MAX_USUARIOS]) protegido por un mutex, que actúa como la única fuente de verdad sobre el estado del sistema.

struct thread_args: agrupa los datos que main necesita pasar a cada hilo recién creado, ya que pthread_create solo admite un único argumento. Contiene el socket del cliente aceptado (client_sock), la dirección del cliente obtenida en el accept (client_dir, de tipo sockaddr_in), y el handle del cliente RPC (client_rpc, de tipo CLIENT*). Se reserva dinámicamente antes de crear cada hilo y el propio hilo es responsable de liberarla al terminar.

1.1 Servidor (server.c)

Las principales funciones implementadas en el archivo server.c, aunque presentan diferencias, siguen la misma estructura general:

1. Adquirir el mutex para acceder de forma segura al estado compartido.
2. Validar las condiciones necesarias para la operación (usuario existente, estado correcto, etc.).
3. Modificar el estado global (clientes[], num_clientes) según corresponda.
4. Liberar el mutex y responder al cliente con un byte de estado (0 = éxito, 1/2/3 = error).
5. Si procede, realizar operaciones adicionales fuera del mutex (conexiones salientes, envío de mensajes).

Todas las funciones con dicha estructura incluyen manejo de errores de comunicación, de forma que ante cualquier fallo se envía el código de error correspondiente y se libera el mutex en caso de estar adquirido.

Funciones principales:

register / unregister: sirven para registrar y dar de baja a un usuario en el sistema, respectivamente. **register_user** comprueba que no exista otro usuario con el mismo nombre y que no se haya alcanzado el límite de usuarios antes de añadirlo al array. **unregister_user** elimina al usuario del array sustituyendo su entrada por la del último elemento, manteniendo el array compacto.

connect_user: marca al usuario como conectado, recibe el puerto de escucha del cliente y guarda su IP y puerto en la estructura correspondiente. Tras responder con éxito, recorre la cola de mensajes pendientes del usuario y los entrega uno a uno abriendo una conexión directa al cliente, notificando además al remitente de cada mensaje con un SEND_MESS_ACK.

disconnect_user: verifica que el usuario esté registrado, conectado y que la IP desde la que se desconecta coincida con la IP registrada en el sistema. En caso afirmativo, limpia la IP y el puerto y marca al usuario como desconectado.

send_message: gestiona tanto el envío de mensajes de texto simples (TIPO=0) como mensajes con fichero adjunto (TIPO=1). Valida que remitente y destinatario existan, asigna un identificador único al mensaje incrementando el contador del remitente, y almacena el mensaje en la cola de pendientes del destinatario. Si el destinatario está conectado, lo entrega de inmediato abriendo una conexión directa a su socket de escucha y, a continuación, notifica al remitente con SEND_MESS_ACK o SEND_MESS_ATTACH_ACK según el tipo.

list_users: comprueba que el solicitante esté registrado y conectado, y devuelve el número de usuarios actualmente conectados seguido de la información de cada uno en formato nombre :: IP :: PUERTO.

Funciones Auxiliares

Para gestionar estas funcionalidades, se han implementado algunas funciones adicionales encargadas de tareas concretas y de mejorar la legibilidad del código:

leer_nombre_usuario: reserva memoria dinámica y lee del socket un string de hasta MAX_NOMBRE caracteres. Devuelve el puntero al nombre leído.

find_user_by_name: realiza una búsqueda sobre el array clientes[] comparando por nombre. Devuelve el índice del usuario encontrado o -1 si no existe. Es utilizada por todas las funciones principales para localizar usuarios antes de operar sobre ellos.

manejador_sigint: instalado como manejador de la señal SIGINT, cierra el socket del servidor y termina el proceso de forma limpia al recibir Ctrl+C.

handle_client: función de entrada de cada hilo creado por main. Lee el tipo de operación enviado por el cliente y el nombre de usuario, y redirige a la función correspondiente. Tras cada operación, invoca **imprimir_log_1** vía RPC para registrar la acción en el servidor de log remoto.

1.2 Cliente (client.py)

El cliente se ha implementado en Python y es multihilo. El hilo principal del programa se encarga de gestionar la interfaz de usuario para enviar las operaciones al servidor, mientras que un segundo hilo de escucha se encarga de recibir mensajes entrantes del servidor para poder procesarlas.

Funciones Principales

Las principales funciones implementadas en el archivo **client.py**, aunque presentan diferencias, siguen la misma estructura general:

1. Conectarse al servidor.
2. Enviar la operación correspondiente al servidor. Dependiendo de la operación, se envía una u otra información.
3. Recibir el resultado (1 byte) de la operación hecha por el servidor.
4. Procesar el resultado recibido, según el que sea, la función imprime lo que corresponda por pantalla, así como devuelve el código de estado que deba.
5. Cerrar la conexión con el servidor.

Todas las funciones con dicha estructura, incluyen manejo de errores mediante bloques **try/except**, de forma que ante cualquier fallo de comunicación se muestra el mensaje de error correspondiente. Estas funciones son las siguientes:

- **register/unregister:** sirven para registrar y dar de baja a un usuario en el servidor, respectivamente.

- **connect:** busca un puerto libre, crea un hilo de escucha del lado del cliente y envía al servidor la solicitud de conexión de un usuario junto con el puerto seleccionado.

- **disconnect:** notifica al servidor de la desconexión de un usuario y cierra el socket de escucha, deteniendo el hilo de recepción de mensajes.

- **send:** normaliza el mensaje a transmitir mediante el servicio web y lo envía al servidor junto con el nombre del remitente y el destinatario.

- **sendAttach:** realiza lo mismo que la función **send**, pero incluyendo además el nombre del fichero adjunto. **Nota importante:** hemos utilizado **SENDATTACH <username> <filename> <message>** ya que es lo que aparece en el código, suponemos que en enunciado aparece **SENDATTACH <username> <message> <filename>** como una errata (el funcionamiento sería el mismo, solo cambia el orden al escribirlo).

- **users:** solicita al servidor la lista de usuarios conectados, los cuales el servidor devuelve junto con la dirección IP y puerto de cada usuario, los cual el cliente almacena en su atributo diccionario **_users_info** para su posterior uso en **getfile**.

- **getfile:** solicita la transferencia de un fichero directamente al cliente que lo posee, sin pasar por el servidor. Para ello, consulta la información del usuario poseedor del archivo en **_users_info** (si no la encuentra, ejecuta **users** para refrescar la información), se conecta al puerto de escucha del cliente e indica el nombre del fichero que quiere recibir.

Funciones Auxiliares

Para gestionar estas funcionalidades, se ha visto conveniente crear algunas funciones adicionales a las que pedía el enunciado para encargarse de algunas tareas y mejorar la lectura del código:

- **_normalizar_mensaje:** sirve para normalizar los espacios del mensaje recibido por un usuario, mediante la conexión al servicio web implementado en el archivo **ws-conversor.py**. Si el servicio no está disponible, simplemente devuelve el mensaje sin modificar (no da error).

- **_recv_string:** se encarga de leer los datos que lleguen al socket pasado como argumento hasta que encuentra el símbolo **‘\0’**. Es una función auxiliar separada para mejorar la legibilidad del código.

- **_listen_messages:** función del **hilo de escucha** que se encarga de aceptar conexiones entrantes del **servidor** u **otro usuario** y procesar los mensajes recibidos según la operación indicada: ***SEND_MESSAGE*** y ***SEND_MESS_ACK*** para

mensajes de texto, ***SEND_MESSAGE_ATTACH*** y ***SEND_MESS_ATTACH_ACK*** para mensajes con fichero adjunto, y ***GET_FILE*** para atender solicitudes de transferencia de ficheros de otros clientes.

1.3 Servicio Web (ws-conversor.py)

El servicio web ha sido implementado en Python, utilizando la librería **Spyne** con el protocolo **SOAP**.

Este servicio expone una única operación a la que hemos llamado **normalizar**, la cual se encarga de recibir un mensaje de texto (string), y devolver el mismo mensaje con espacios repetidos eliminados, de forma que las palabras queden separadas por un único espacio en blanco. Para ello, se utiliza la expresión regular **re.sub(r' +', ' ', mensaje).strip()**, mediante la cual se sustituyen espacios en blanco seguidos, por un único espacio.

El servicio se despliega en la **máquina local** del cliente en el **puerto 8000**. En las funciones que requieren de dicho servicio web (**send** y **sendAttach**, como hemos visto en el apartado anterior), el cliente accede al servicio mediante la librería **Zeep**, descargando automáticamente el **WSDL** desde **http://localhost:8000/?wsdl**. En caso de que el servicio no esté disponible, simplemente deja el mensaje original y se envía a su destino.

1.4 RPC

La implementación se divide en dos partes claramente diferenciadas: el fichero de definición de la interfaz (**plantilla.x**) y el código de la lógica de negocio (**log_server.c**), siendo el resto de ficheros generados automáticamente por la herramienta rpcgen a partir del primero.

El fichero .x es el punto de partida del servidor RPC, define un único programa llamado SERVIDOR_LOG con número de identificación 100001, versión VERSION_1, y un único procedimiento:

int imprimir_log(string usuario<256>, string operacion<256>) = 1;

Este procedimiento recibe el nombre del usuario y la operación realizada, ambos strings de hasta 256 caracteres, y devuelve un entero indicando el resultado. A partir de este fichero, rpcgen genera automáticamente el resto de la infraestructura.

Funciones Implementadas:

log_server.c es el único fichero escrito manualmente. Contiene exclusivamente la lógica de negocio del servidor de log, separada completamente de la infraestructura de comunicación:

imprimir_log_1_svc: es la función de implementación del procedimiento RPC. Recibe el nombre del usuario, la operación y un puntero al resultado, e imprime ambos separados por un tabulador. Establece *resultado = 0 para indicar éxito al cliente, llama a fflush(stdout) para garantizar que el log se escribe inmediatamente sin quedar retenido en el buffer, y devuelve TRUE para que el dispatcher envíe la respuesta.

servidor_log_1_freeresult: función de limpieza requerida por el framework RPC, su presencia es obligatoria para que el servidor funcione correctamente.

2. Compilación y ejecución

2.1 Compilación del servidor

El servidor (escrito en C), se compila mediante el **Makefile** proporcionado. Los comandos disponibles son los siguientes:

make → Compila el servidor principal y el servidor RPC y genera los ejecutables en **build/**.

make clean → Elimina el directorio **build/**, por tanto, todos los ejecutables generados.

2.2 Ejecución del sistema

Para ejecutar el sistema completo es necesario arrancar varios procesos en el siguiente orden:

1. **Servidor RPC:**
   ```
   ./build/servidor_rpc
   ```

2. **Servicio Web:**
   ```
   python3 ws-conversor.py
   ```

3. **Servidor de Mensajería:**

   ```
   export LOG_RPC_IP= IP
   ./build/server -p <puerto>
   ```

4. **Clientes** (uno por terminal, para cada cliente a probar):
   ```
   python3 client/client.py -s <IP_servidor> -p <puerto>
   ```

Una vez arrancados los procesos correspondientes, la consola del cliente acepta los siguientes comandos a realizar:

- **REGISTER** <userName> / **UNREGISTER** <userName>
- **CONNECT** <userName> / **DISCONNECT** <userName>

- **USERS**
- **SEND** <userName> <message>
- **SENDATTACH** <username> <fileName> <message>
- **GETFILE** <userName> <remoteFileName> <localFileName>
- **QUIT**

---

3. Batería de Pruebas y Resultados

P1 — REGISTER / UNREGISTER

Objetivo: Verificar registro de usuarios, detección de duplicados, baja con borrado de mensajes pendientes y manejo de errores.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
1	Registrar usuario nuevo	REGISTER alice REGISTER bob	REGISTER alice OK REGISTER bob OK	**OK**	
2	Registrar nombre duplicado	REGISTER alice	USERNAME IN USE (cod 1)	**OK**	
3	Registrar 100 usuarios (llenar array)	REGISTER u1 ... u100	u1..u100 OK u101: STORAGE FULL (cod 2)	**OK**	MAX_USUARIOS=100. u101 falla con cod 2.
4	Enviar msg a offline, luego UNREGISTER	SEND bob "hola" (bob offline) UNREGISTER bob	STORED + UNREGISTER OK Al reconectar bob no tiene pendientes	**OK**	UNREGISTER borra la cola.
5	UNREGISTER usuario inexistente	UNREGISTER carol	USER DOES NOT EXIST (cod 1)	**OK**	

P2 — CONNECT / DISCONNECT

Objetivo: Verificar conexión y desconexión, validación de IP, entrega de mensajes almacenados y notificación ACK al remitente.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
6	CONNECT usuario registrado offline	CONNECT alice	CONNECT OK (cod 0)	**OK**	
7	CONNECT usuario no registrado	CONNECT carol	CONNECT FAIL, USER DOES NOT EXIST (cod 1)	**OK**	
8	CONNECT usuario ya conectado	CONNECT alice (de nuevo)	USER ALREADY CONNECTED (cod 2)	**OK**	
9	3 msgs a offline, luego CONNECT	SEND bob ms1 SEND bob ms2 SEND bob ms3 CONNECT bob	Bob recibe 3 SEND_MESSAGE Alice recibe 3 SEND_MESS_ACK	**OK**	Entrega ordenada de pendientes.
10	DISCONNECT usuario conectado	DISCONNECT alice	DISCONNECT OK (cod 0)	**OK**	
11	DISCONNECT no conectado	DISCONNECT alice (de nuevo)	DISCONNECT FAIL, USER NOT CONNECTED (cod 2)	**OK**	Caso de error
12	DISCONNECT usuario inexistente	DISCONNECT carol	DISCONNECT FAIL, USER DOES NOT EXIST (cod 1)	**OK**	Caso de error
13	DISCONNECT desde IP distinta	Conectar alice desde IP A DISCONNECT alice desde IP B	DISCONNECT FAIL (cod 3)	**N/A**	Caso de error

P3 — SEND con destinatario conectado

Objetivo: Verificar entrega inmediata de mensajes entre usuarios conectados y confirmación ACK al remitente.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
14	A envia a B estando ambos conectados	SEND bob "Hola Bob"	Bob recibe el mensaje Alice recibe SEND MESSAGE id OK	**OK**	Entrega en tiempo real + ACK.
15	Envio triangular A->B->C->A	SEND bob msg1 SEND carol msg2 SEND alice msg3	Cada uno recibe su mensaje con ID secuencial	**OK**	Tres clientes conectados.
16	SEND a usuario inexistente	SEND carol "hola"	SEND FAIL, USER DOES NOT EXIST (cod 1)	**OK**	Caso de error

P4 — USERS / CONNECTED USERS

Objetivo: Verificar listado de usuarios conectados con formato nombre :: IP :: puerto.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
17	USERS con 3 de 5 conectados	REGISTER u1..u5 CONNECT u1,u2,u3 USERS (desde u1)	CONNECTED USERS (3 users connected) OK u1 u2 u3	**OK**	Lista los conectados con formato nombre :: IP :: puerto.
18	USERS desde usuario no conectado	USERS (desde u4 offline)	CONNECTED USERS FAIL, USER IS NOT CONNECTED (cod 1)	**OK**	Caso de error
19	USERS desde usuario no registrado	USERS (desde carol)	CONNECTED USERS FAIL (cod 2)	**OK**	Caso de error

P5 — Concurrencia: múltiples clientes simultáneos

Objetivo: Verificar que el servidor maneja múltiples hilos concurrentes sin deadlocks ni inconsistencias en IDs.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
20	10 clientes conectados simultaneamente	10x REGISTER + 10x CONNECT	Todos OK, sin errores de conexión	**OK**	Servidor maneja 10 hilos.
21	5 envian en paralelo a los otros 5	5 clientes lanzan SEND concurrente	Todos reciben sus mensajes IDs secuenciales por remitente Todos los ACK correctos	**OK**	
22	100 SEND entre 10 usuarios concurrentes	100 SEND ejecutados en paralelo	Sin deadlocks ni bloqueos Servidor responde a todas las peticiones	**OK**	Test de estrés superado.

P6 — SENDATTACH + GETFILE

Objetivo: Verificar envío de mensajes con archivo adjunto y transferencia P2P del fichero.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
23	SENDATTACH entre usuarios conectados	SENDATTACH bob f.txt "Mira el archivo"	Bob recibe SEND_MESSAGE_ATTACH Alice recibe SEND_MESS_ATTACH_ACK	**OK**	Entrega inmediata con fichero.
24	GETFILE: transferencia de fichero texto	GETFILE alice f.txt copia.txt	copia.txt = f.txt Mismo tamano byte a byte	**OK**	Transferencia P2P correcta.
25	GETFILE con fichero binario (imagen)	SENDATTACH bob img.png "foto" GETFILE alice img.png local.png	local.png = img.png diff sin diferencias	**OK**	1 KB binario sin corrupción.

P7 — SENDATTACH con destinatario offline

Objetivo: Verificar que SENDATTACH almacena el mensaje con nombre de fichero y lo entrega al conectar.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
26	SENDATTACH a usuario offline	SENDATTACH bob notas.txt "Apuntes" (bob offline)	Servidor: MESSAGE STORED Alice recibe SENDATTACH OK	**OK**	
27	Destinatario se conecta y recibe	CONNECT bob	Bob recibe SEND_MESSAGE_ATTACH con remitente, id, msg y notas.txt	**OK**	
28	GETFILE tras reconexion	GETFILE alice notas.txt mis_notas.txt	Transferencia correcta del fichero adjunto	**OK**	

P8 — Normalización de mensajes (Web Service SOAP)

Objetivo: Verificar que el cliente normaliza los mensajes mediante el servicio web SOAP antes de enviarlos al servidor.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
29	Mensaje con espacios múltiples	SEND bob "Hola mundo cruel"	Bob recibe: "Hola mundo cruel"	**OK**	Normalización SOAP funciona.
30	Mensaje con espacios al inicio/fin	SEND bob " hola "	Bob recibe: "hola"	**OK**	strip() aplicado correctamente.
31	Servicio web caido	Detener servicio SOAP SEND bob "Hola mundo"	Bob recibe "Hola mundo" sin normalizar	**OK**	Caso de error

P9 — RPC Logging y tolerancia a fallos

Objetivo: Verificar que las operaciones se registran en el servidor RPC y que el sistema tolera la caída del servidor de log.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
32	Registro normal de operaciones en RPC	REGISTER alice REGISTER bob CONNECT alice CONNECT bob SEND bob "hola"	Servidor RPC registra: alice REGISTER bob REGISTER alice CONNECT bob CONNECT alice SEND	**OK**	
33	Tolerancia a caida del servidor RPC	Matar servidor_log SEND bob "msg2"	Servidor de mensajeria imprime error RPC en stderr Sigue funcionando con normalidad	**FAIL**	Caso de error: error, pero como no se pide explícitamente **decidimos dejar este funcionamiento.**
34	Recuperación tras reconexion RPC	Relanzar servidor_log SEND bob "msg3"	msg3 se registra normalmente en el log RPC	**OK**	Tras reiniciar ambos servidores, el log vuelve a funcionar.

P10 — Robustez y casos límite

Objetivo: Verificar comportamiento ante condiciones extremas: límite de almacenamiento, overflow de IDs, caída del servidor y desconexiones.

N°	Acción	Entrada	Salida esperada	OK/FAIL	Observaciones
35	Cola de pendientes llena (MAX_MENSAJES=200)	Enviar 201 SEND a usuario offline	Mensajes 1-200: STORED Mensaje 201: SEND FAIL (cod 2)	**OK**	Cola limitada correctamente a 200 mensajes.
36	Overflow del contador de ID (aprox.)	Enviar 30 mensajes y verificar IDs	IDs secuenciales sin huecos Rango: 1..30	**OK**	Contador de mensajes funciona correctamente.
37	Ctrl+C en servidor (SIGINT)	Servidor recibe SIGINT Cliente intenta REGISTER	Servidor cierra socket limpiamente Cliente: REGISTER FAIL	**OK**	Cierre limpio con manejador SIGINT.
38	Cliente se desconecta durante envio	Destinatario conectado Cerrar su socket antes de SEND Remitente envía mensaje	Servidor marca destinatario como desconectado Mensaje queda STORED	**OK**	El servidor detecta el fallo de conexión y almacena el mensaje.

---

4. Conclusiones

El desarrollo de esta práctica nos ha permitido trabajar con tres tecnologías de comunicación distintas en una misma aplicación: sockets TCP en C, servicios web SOAP en Python y RPC con ONC-RPC en C. Combinarlas en un mismo proyecto ha sido útil para entender cuándo y por qué se usa cada una.

La parte más complicada fue la gestión de la concurrencia en el servidor, especialmente el uso correcto del mutex para evitar condiciones de carrera al acceder al estado compartido desde múltiples hilos a la vez. Gracias a las prácticas anteriores de la asignatura, ya teníamos cierta base que nos ayudó en esta parte. El resto del código del servidor, como las funciones de mensajería en C o el RPC, también resultó familiar por la misma razón. Lo más novedoso e interesante fue el lado del cliente, ya que implementarlo en Python con hilos concurrentes fue una experiencia bastante diferente a lo que habíamos hecho hasta ahora.

En la Parte 2, el principal problema fue la incompatibilidad de Spyne con Python 3.12, que obligó a parchear manualmente varios ficheros internos de la librería. Aún así, hemos podido ver el funcionamiento de conectar nuestra aplicación a un servidor web, que, pese a que no tiene mucha importancia su funcionamiento en nuestra práctica, es un buen acercamiento para ver cómo utilizar estos servicios.

En general, ha sido una práctica interesante que nos ha ayudado a entender cómo funcionan por dentro los sistemas distribuidos reales, más allá de la teoría.